Grimlock: Guarding High-Agency Systems with eBPF and Attested Channels

Anonymous Author(s)

Abstract

Agentic systems increasingly run user-authored orchestration code that invokes tools, spawns subtasks, and delegates work across machines and clouds, creating a tension between high agency and operator requirements for identity, authorization, provenance, and auditable scope. We present *Grimlock*, an *Agent Guard* that restores separation of concerns by enforcing trust in the sandbox substrate while leaving user code unchanged.

Grimlock combines *eBPF-enforced traffic interception* with *post-handshake attestation bound to kTLS channels*. eBPF redirects sandbox ingress/egress to a user-space proxy, which uses standard TLS 1.3 channel bindings (exporters / exported authenticators) to authorize an attested channel and mint short-lived, channel-bound *scope tokens*. At the destination sandbox, identity and scope are re-validated; TLS is terminated; and plaintext is released only after policy checks.

Grimlock is transparent to agents, all enforcement is confined to the sandbox layer via commodity Linux primitives, enabling uniform identity and scope-bound delegation across heterogeneous multi-cloud environments.

CCS Concepts: • **Computer systems organization** → *Cloud computing*; • **Security and privacy** → *Network security*; **Systems security**.

Keywords: agent OS, high agency, sandboxing, eBPF, attested TLS, post-handshake attestation, remote attestation, confidential computing, workload identity, provenance, service mesh, multi-cloud

ACM Reference Format:
Anonymous Author(s). 2026. Grimlock: Guarding High-Agency Systems with eBPF and Attested Channels. In *Proceedings of The 1st Workshop on Operating Systems Design for AI Agents (AgenticOS), co-located with ASPLOS 2026 (AgenticOS '26).* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Agentic software increasingly acts as an orchestrator: it plans, calls tools, invokes other agents, and composes workflows across machines and clouds [2, 3, 7, 9]. This yields a desirable property we call **high agency**—the user layer expresses intent without micromanaging infrastructure. But high agency often collapses security layering: application code embeds identity and authorization logic, duplicates policy checks, and stitches ad-hoc credentials, reintroducing classic confused-deputy and privilege-management pitfalls [1, 4]. The result is brittle trust, weak auditability, and poor portability.

We argue an Agentic OS should provide high agency *and* strict trust boundaries by separating concerns: user code handles orchestration, while a higher-privilege sandbox substrate enforces identity, authentication, authorization, provenance. **Grimlock** realizes this split with eBPF no-bypass mediation, and kTLS-bound post-handshake attestation for authenticated agent-to-agent communication.

Why eBPF. High-agency agents are untrusted and highly dynamic: they may spawn processes, open arbitrary sockets, and call tools that live outside the application address space. Purely library- or runtime-level enforcement is therefore easy to bypass. Grimlock uses eBPF because it enables *OS-enforced, application-transparent* mediation at the sandbox boundary: we can interpose on ingress/egress, attach stable identities to flows, and *force* all traffic through a controlled path (e.g., redirecting sockets to a guard proxy) without modifying agent code.

Why post-handshake attestation. Attested TLS can be categorized by when evidence is generated relative to connection establishment [5]. We adopt post-handshake attestation because it preserves interoperability with commodity TLS 1.3 stacks (no handshake modifications) while still binding evidence to the established channel using exporters/EKM and freshness nonces, mitigating replay, diversion, and relay [5, 6, 8].

We prototype Grimlock, an Agentic OS guard layer that transparently secures agent communication by enforcing a guarded, no-bypass datapath and channel-bound authentication. Contributions of this paper include:

- **Seamless routing:** transparent interception and routing of all sandbox ingress/egress.
- **No-bypass enforcement:** eBPF-based mediation that prevents agents from escaping the guard.
- **Kernel TLS dataplane:** a kTLS-based TLS 1.3 datapath with record processing in kernel space.
- **Authenticated A2A protocol:** a post-handshake, channel-bound attestation protocol for secure agent-to-agent communication.

2 Design of Grimlock

Grimlock provides substrate-level security for high-agency agents by enforcing a no-bypass network mediation point

AgenticOS '26, Pittsburgh, PA, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

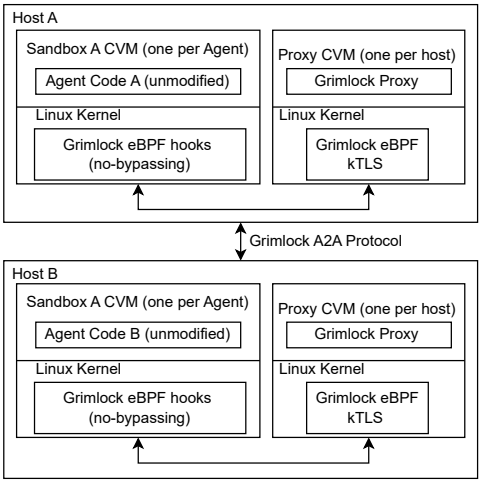

Figure 1. Grimlock deployment. Each host runs a per-agent sandbox Confidential VM (CVM) and a per-host proxy CVM. eBPF enforces no-bypass mediation and steers traffic through the proxy. Proxies communicate via the Grimlock A2A Protocol over TLS 1.3 (kTLS-assisted datapath).

at the sandbox boundary and binding authorization to live, encrypted channels, shown in Figure. 1.

2.1 Threat Model

We assume an active network adversary capable of eavesdropping, replay, relay/diversion, and man-in-the-middle attacks. We also consider a malicious or buggy agent runtime that may attempt to bypass mediation (e.g., direct sockets, alternate stacks) or escalate scope.

We do *not* trust user-layer code to correctly implement identity or authorization, and we treat the host network and surrounding infrastructure as untrusted. Agent code is produced once, verified at build time, and deployed as an immutable artifact.

Grimlock targets three properties: (i) **no-bypass**, all sandbox traffic traverses the guard; (ii) **channel binding**, authorization artifacts are non-transferable across channels; and (iii) **least privilege**, delegation propagates auditable, scoped permissions.

2.2 Architecture and Data Path

Grimlock comprises per-agent sandboxes (CVMs) and a per-host guard proxy. Each agent executes inside a confidential, attestable CVM; the CVM is trusted only after remote attestation succeeds under operator policy, covering the boot chain and measured Grimlock substrate (guest OS and kTLS).

Grimlock enforces mandatory mediation at the sandbox boundary. eBPF hooks interpose on all sandbox ingress/egress and redirect every flow to the local guard proxy CVM, preventing agents from bypassing policy. The proxy completes a standard TLS 1.3 handshake (using kTLS for record processing) and then runs post-handshake attestation bound to the live channel via exporter-derived channel bindings. Upon successful appraisal by a verifier/issuer, policy mints a short-lived, channel-bound *Scope Token* encoding least-privilege delegation. Traffic is forwarded to the destination host, where the receiving guard re-validates evidence, token scope, and channel binding; terminates TLS; and releases plaintext to the destination sandbox only after policy checks succeed.

2.3 Grimlock A2A Protocol and Scope Token

Agent A initiates A2A communication by issuing a standard socket `connect()` from within its sandbox. Grimlock enforces *no-bypass* at the sandbox boundary using eBPF TC. The guard maintains per-flow state (6-tuple → sandbox image ID, source and destination ip, destination port, requested-scope hash, expiry) and forwards control traffic over a long-lived host-to-host channel, amortizing setup and allowing application TCP establishment to overlap with guard authentication and authorization. Over the control channel, guards run kTLS(TLS 1.3) to derive traffic secrets for the intercepted socket. The first contact with a peer performs full authentication and caches a node-level authentication context; subsequent connections reuse this cached context. After key derivation, each guard injects the resulting TLS keys into the intercepted socket using kTLS (TX/RX where supported), shifting record encryption/decryption to the kernel fast path without modifying agent code.

Authorization is enforced as a *post-handshake* gate bound to the live channel. Immediately after the TLS handshake completes (before releasing any application payload), guards compute a channel binding $\mathsf{cb} = \mathsf{Exporter}(\mathsf{TLS}, \mathsf{label}, \mathsf{ctx})$ where ctx includes a fresh nonce, destination audience, and the requested delegation scope. The responder returns TEE evidence (quote/report plus measured identifiers for the CVM and userspace components) that commits to $H(\mathsf{cb})$; a verifier/issuer appraises the evidence under operator policy and mints a short-lived *TEE Scope Token* encoding $\mathsf{sub}/\mathsf{aud}/\mathsf{scope}$ and $\mathsf{cbh}{=}H(\mathsf{cb})$. The receiving guard re-validates evidence, token signature/scope/audience/expiry, and exporter binding; only then does it terminate TLS and release plaintext into the destination sandbox, ensuring channel-bound, least-privilege delegation and preventing replay/relay across connections.

3 Conclusion

High-agency agentic software demands both flexibility and strong trust boundaries. Grimlock delivers this separation by enforcing no-bypass traffic mediation with eBPF and securing agent-to-agent communication with kTLS (TLS 1.3) channels.

References

[1] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38.

[2] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. International Conference on Learning Representations, ICLR.

[3] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688* (2023).

[4] Ravi S. Sandhu and Pierangela Samarati. 1994. Access Control: Principles and Practice. *IEEE Communications Magazine* 32 (1994), 40–48.

[5] Muhammad Usama Sardar. 2026. Pre-, Intra- and Post-handshake Attestation. Internet-Draft, IETF SEAT WG. Work in Progress: draft-usama-seat-intra-vs-post-02.

[6] Nick Sullivan. 2022. Exported Authenticators in TLS. RFC 9261. doi:10.17487/RFC9261

[7] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: Survey, landscape, and vision. *Automated Software Engineering* 32, 2 (2025), 1–36.

[8] Samuel Whited. 2022. Channel Bindings for TLS 1.3. RFC 9266. doi:10.17487/RFC9266

[9] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.